

Практическая работа «Реализация цифровых методов кодирования изображений»

Цель: изучить и реализовать основные цифровые методы кодирования изображений.

Теоретическая часть

В видеопамяти находится двоичная информация об изображении, выводимом на экран. В зависимости от способа формирования изображений компьютерную графику принято подразделять на растровую, векторную. Отдельным направлением считается трехмерная (3D) графика, изучающая приемы и методы построения объемных моделей объектов в виртуальном пространстве. В ней сочетаются векторный и растровый способы формирования изображений.

Форму представления на экране дисплея графического изображения, состоящего из отдельных точек (пикселей), называют растровой. Минимальным объектом в растровом графическом редакторе является точка (пиксель). Растровые изображения представляют собой однослойную сетку точек, называемых пикселями (**pixel**, от англ. **picture element**). Код пикселя содержит информацию о его цвете.

Для черно-белого изображения (без полутонов) пиксель может принимать только два значения: белый и черный (светится – не светится), а для его кодирования достаточно одного бита памяти: *1 – белый, 0 – черный*.

Пиксель на цветном дисплее может иметь различную окраску, поэтому одного бита на пиксель недостаточно. Для кодирования 4-цветного изображения требуются два бита на пиксель, поскольку два бита могут принимать 4 различных состояния. Может использоваться, например, такой вариант кодировки цветов: *00 – черный, 10 – зеленый, 01 – красный, 11 – коричневый*.

Цветовые модели расположены в трехмерной системе координат, образующей цветовое пространство, так как из законов Грассмана следует, что цвет можно выразить точкой в трехмерном пространстве.

Первый закон Грассмана (закон трехмерности). Любой цвет однозначно выражается тремя составляющими, если они линейно независимы. Линейная независимость заключается в невозможности получить любой из этих трех цветов сложением двух остальных.

Второй закон Грассмана (закон непрерывности). При непрерывном изменении излучения цвет смеси также меняется непрерывно. Не существует такого цвета, к которому нельзя было бы подобрать бесконечно близкий.

Третий закон Грассмана (закон аддитивности). Цвет смеси излучений зависит только от их цвета, но не спектрального состава. То есть цвет (**C**) смеси выражается суммой цветовых уравнений излучений.

Любой цвет (**C**) может быть выражен в цветовом пространстве вектором, который описывается уравнением: которое практически идентично уравнению свободного вектора в пространстве, рассматриваемому в векторной математике. Направление вектора характеризует цветность, а его модуль выражает яркость.

В основе кодирования цветной графики лежит принцип декомпозиции произвольного цвета на основные составляющие. Применяют несколько систем кодирования. Рассмотрим кратко основные цветовые модели.

1. Принцип метода **RGB** заключается в следующем: известно, что любой цвет можно представить в виде комбинации трех цветов: красного (**Red, R**), зеленого (**Green, G**),

синего (**Blue, B**). Другие цвета и их оттенки получаются за счет наличия или отсутствия этих составляющих. По первым буквам основных цветов система и получила свое название – **RGB**. Данная цветовая модель является аддитивной, то есть любой цвет можно получить сочетанием основных цветов в различных пропорциях. При наложении одного компонента основного цвета на другой яркость суммарного излучения увеличивается. Если совместить все три компонента, то получим ахроматический серый цвет, при увеличении яркости которого происходит приближение к белому цвету.

При **256** градациях тона (каждая точка кодируется 3 байтами) минимальные значения **RGB (0,0,0)** соответствуют черному цвету, а белому – максимальные с координатами **(255, 255, 255)**. Чем больше значение байта цветовой составляющей, тем этот цвет ярче. Например, темно-синий кодируется тремя байтами **(0, 0, 128)**, а ярко-синий **(0, 0, 255)**.

2. Модель **HSB** характеризуется тремя компонентами: оттенок цвета (Hue), насыщенность цвета (Saturation) и яркость цвета (Brightness). Можно получить большое количество произвольных цветов, регулируя эти компоненты. Эту цветовую модель лучше применять в тех графических редакторах, в которых изображения создают сами, а не обрабатывают уже готовые. Значение цвета выбирается как вектор, выходящий из центра окружности. Направление вектора задается в угловых градусах и определяет цветовой оттенок. Насыщенность цвета определяется длиной вектора, а яркость цвета задается на отдельной оси, нулевая точка которой имеет черный цвет. Точка в центре соответствует белому (нейтральному) цвету, а точки по периметру – чистым цветам.

3. Принцип метода **СМУК**. Эта цветовая модель используется при подготовке публикаций к печати. Каждому из основных цветов ставится в соответствие дополнительный цвет (дополняющий основной до белого). Получают дополнительный цвет за счет суммирования пары остальных основных цветов. Значит, дополнительными цветами для красного является голубой (*Cyan, C*) = *зеленый + синий = белый – красный*, для зеленого – пурпурный (*Magenta, M*) = *красный + синий = белый – зеленый*, для синего – желтый (*Yellow, Y*) = *красный + зеленый = белый – синий*. Причем принцип декомпозиции произвольного цвета на составляющие можно применять как для основных, так и для дополнительных, то есть любой цвет можно представить или в виде суммы красной, зеленой, синей составляющей или же в виде суммы голубой, пурпурной, желтой составляющей. В основном такой метод принят в полиграфии. Но там еще используют черный цвет (**Black**, так как буква **B** уже занята синим цветом, то обозначают буквой **K**). Это связано с тем, что наложение друг на друга дополнительных цветов не дает чистого черного цвета.

Различают несколько режимов представления цветной графики:

- а) полноцветный (**True Color**);
- б) **High Color**;
- в) индексный.

При полноцветном режиме для кодирования яркости каждой из составляющих используют по **256** значений (восемь двоичных разрядов), то есть на кодирование цвета одного пикселя (в системе **RGB**) надо затратить $8 \times 3 = 24$ разряда. Это позволяет однозначно определять **16,5 млн цветов**. Это довольно близко к чувствительности человеческого глаза. При кодировании с помощью системы **СМУК** для представления цветной графики надо иметь $8 \times 4 = 32$ двоичных разряда.

Режим **High Color** – это кодирование при помощи 16-разрядных двоичных чисел, то

есть уменьшается количество двоичных разрядов при кодировании каждой точки. Но при этом значительно уменьшается диапазон кодируемых цветов.

При индексном кодировании цвета можно передать всего лишь **256** цветовых оттенков. Каждый цвет кодируется при помощи восьми бит данных. Но так как **256** значений не передают весь диапазон цветов, доступный человеческому глазу, то подразумевается, что к графическим данным прилагается палитра (справочная таблица), без которой воспроизведение будет неадекватным: море может получиться красным, а листья – синими. Сам код точки раstra в данном случае означает не сам по себе цвет, а только его номер (индекс) в палитре. Отсюда и название режима – индексный.

Количество различных цветов K и количество битов для их кодирования b связаны между собой формулой $K = 2^b$. Так, **21 = 2**, **22 = 4**, **23 = 8**, **24 = 16** и т. д. Для получения цветовой гаммы из **256 цветов** требуется 8 битов = 1 байт на каждый пиксель, так как **28 = 256**. Величина b является битовой глубиной цвета, K – количество цветов.

Разрешающая способность монитора (количество точек по горизонтали и вертикали), а также число возможных цветов каждой точки определяются типом монитора. Например: **640×480 = 307 200** точек, **800×600 = 480 000** точек.

Объем памяти, необходимой для хранения графического изображения, занимающего весь экран, равен произведению количества пикселей (разрешающей способности) на число бит, кодирующих одну точку. Объем графического файла в битах определяется как произведение количества пикселей на разрядность цвета (битовую глубину).

Итак, объем памяти, необходимой для хранения графического изображения в битах V рассчитывается по формуле:

$$V = K \cdot I,$$

где K - количества пикселей, хранящих изображение;

I - глубина цвета в битах.

Например, при разрешении **640×480** и количестве цветов **16 (4 бита)** объем памяти равен: **640×480×4=1 228 800 (бит) = 153 600 (байт) = 150 (Кбайт)**.

Если в растровой графике базовым элементом изображения является точка, то в векторной графике - линия. Линия описывается математически как единый объект, и потому объем данных для отображения объекта средствами векторной графики существенно меньше, чем в растровой графике. Линия - элементарный объект векторной графики. Как и любой объект, линия обладает свойствами: формой (прямая, кривая), толщиной, цветом, начертанием (сплошная, пунктирная).

Замкнутые линии приобретают свойство заполнения. Охватываемое ими пространство может быть заполнено другими объектами (текстуры, карты) или выбранным цветом. Простейшая незамкнутая линия ограничена двумя точками, именуемыми узлами или опорными точками. Узлы также имеют свойства, параметры которых влияют на форму конца линии и характер сопряжения с другими объектами. Все прочие объекты векторной графики составляются из линий. Например, куб можно составить из шести связанных прямоугольников, каждый из которых, в свою очередь, образован четырьмя связанными линиями. Возможно представить куб и как двенадцать связанных линий, образующих ребра.

В отличие от растровой графики векторное изображение многослойно. Каждый элемент векторного изображения – линия, прямоугольник, окружность или фрагмент текста – располагается в своем собственном слое, пиксели которого устанавливаются независимо от других слоев, и описывается с помощью специального языка

(математических уравнения линий, дуг, окружностей и т.д.) Сложные объекты (ломаные линии, различные геометрические фигуры) представляются в виде совокупности элементарных графических объектов.

Практическая часть

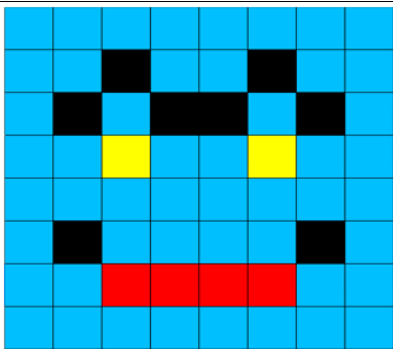
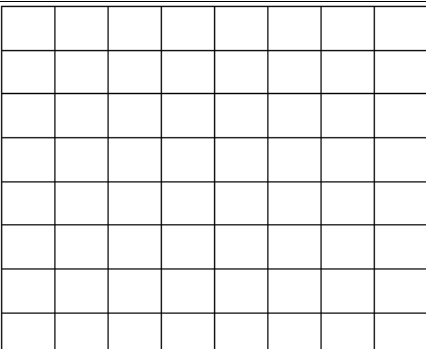
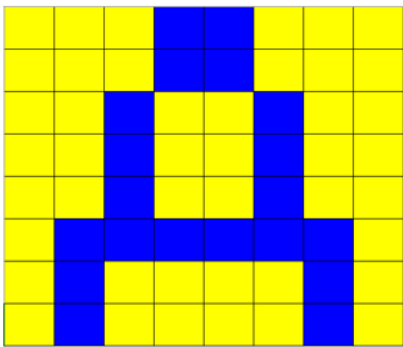
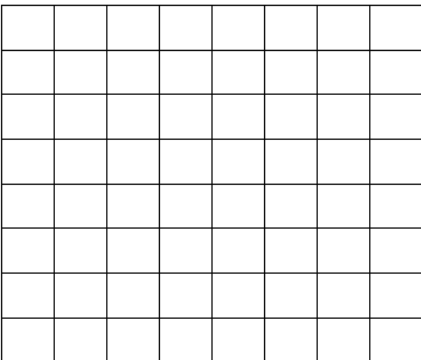
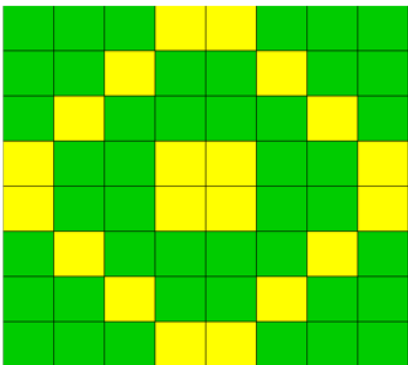
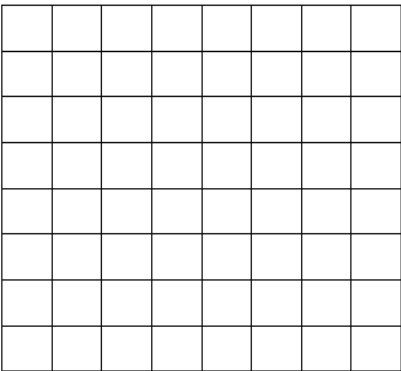
Задание 1. Определить, сколько цветов кодируется при битовой глубине: а) 4, б) 3 в) 5

Задание 2. Определить объем памяти для хранения информации в один пиксель при битовой глубине 4

Задание 3. Определить объем памяти для хранения графической информации 480x240 пикселей и количестве цветов 32

Задание 4. Определить битовую глубину кодирования графической информации, если известно, что для хранения графического файла размером 640×480 пикселей потребовалось 150Кбайт

Задание 5. Закодировать изображение при трехбитном кодировании изображения. (варианты распределить согласно списку группы).

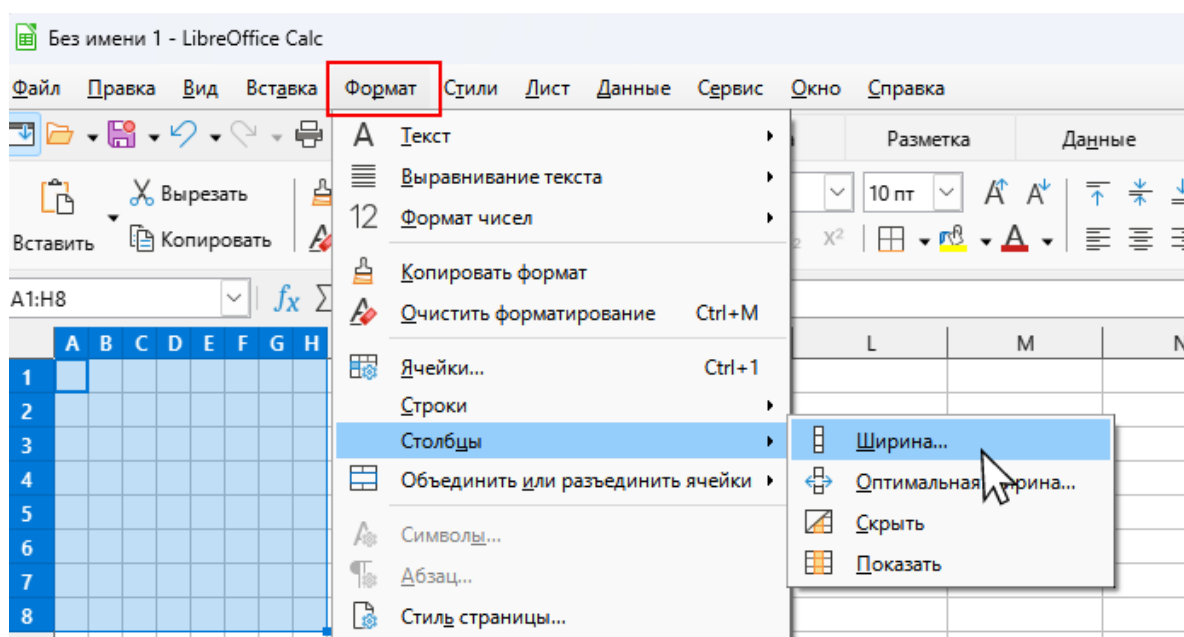
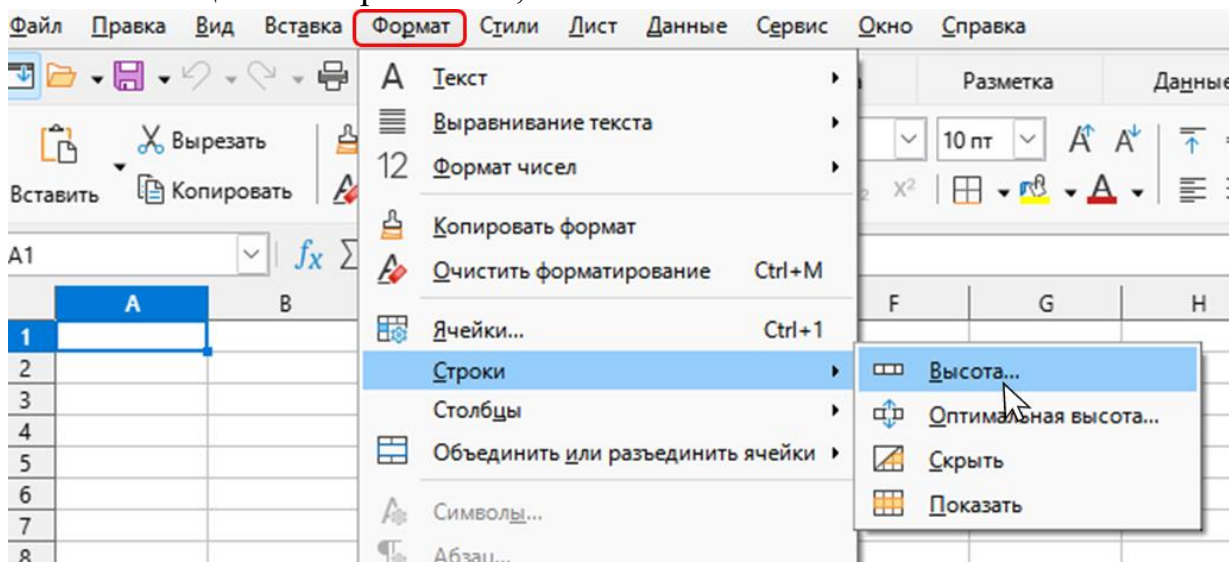
Вариант	Задание	
Вариант 1		
Вариант 2		
Вариант 3		

Задание 6. Нарисовать изображение по закодированной таблице. (в EXCEL, LibreOffice Calc)

*Инструкция для LibreOffice.Calc:

Формат => Строки => Высота => 0,5 см

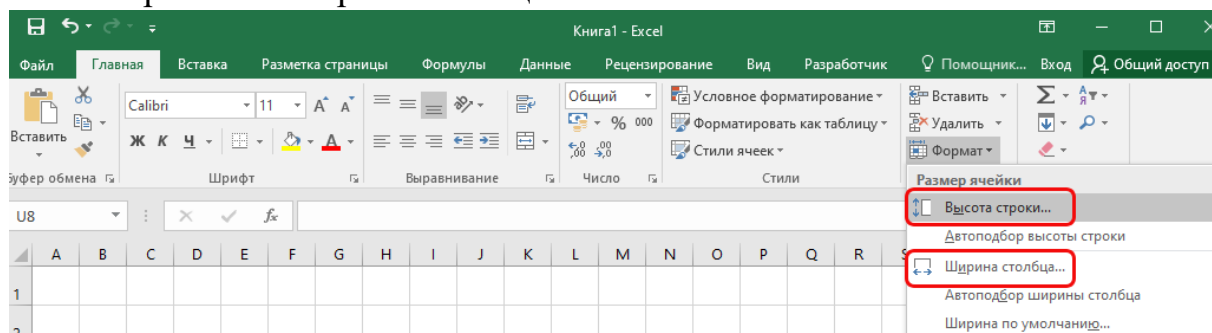
Формат => Столбцы => Ширина => 0,5 см



**Инструкция для MS Excel:*

Главная => Формат => Высота строки => 25

Главная => Формат => Ширина столбца => 5



001	101	101	101	101	101	101	001
101	001	001	001	001	001	001	101
101	001	110	001	001	110	001	101

101	001	001	001	001	001	001	101
101	001	110	001	001	110	001	101
101	001	001	110	110	001	001	101
001	101	001	001	001	001	101	001
001	001	101	101	101	101	001	001

110	000	000	110	110	000	000	110
000	100	100	000	000	100	100	000
000	100	100	100	100	100	100	000
000	100	100	100	100	100	100	000
110	000	100	100	100	100	000	110
110	000	100	100	100	100	000	110
110	110	000	100	100	000	110	110
110	110	110	000	000	110	110	110

011	011	010	010	010	010	011	011
011	011	010	010	010	111	010	011
011	011	110	000	110	000	011	011
011	011	110	110	100	110	011	011
011	010	001	010	010	001	010	011
011	110	001	001	001	001	110	011
011	011	001	001	001	001	011	011
011	011	100	011	011	100	011	011

Задание 7. 265-цветный рисунок содержит 1 Кбайт информации. Из скольких точек он состоит?

Задание 8. Видеопамять имеет объем, в котором может храниться 8-цветное изображение размером 640X350 точек. Какого размера изображение можно хранить в том же объеме видеопамяти, если использовать 512-цветную палитру?

Перечень контрольных вопросов

1. В чем принципиальная разница между аналоговым и дискретным представлением графической информации? Приведите примеры каждого способа.
2. Что такое пространственная дискретизация изображения?

3. От каких параметров зависит качество растрового изображения при его цифровом кодировании?
4. Что такое симметричность алгоритма сжатия? Почему этот параметр важен для практического применения?
5. Почему высокая скорость компрессии, высокое качество изображений и высокая степень компрессии взаимно противоречивы?